

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	6
1.2 Описание выходных данных.....	6
2 МЕТОД РЕШЕНИЯ.....	7
3 ОПИСАНИЕ АЛГОРИТМОВ.....	9
3.1 Алгоритм конструктора класса Character.....	9
3.2 Алгоритм метода change класса Healer.....	9
3.3 Алгоритм функции printValues.....	10
3.4 Алгоритм функции main.....	10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	12
5 КОД ПРОГРАММЫ.....	14
5.1 Файл Character.cpp.....	14
5.2 Файл Character.h.....	14
5.3 Файл Healer.cpp.....	15
5.4 Файл Healer.h.....	15
5.5 Файл main.cpp.....	16
6 ТЕСТИРОВАНИЕ.....	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	18

1 ПОСТАНОВКА ЗАДАЧИ

Разработать программную систему на языке C++, моделирующую объект персонажа игры, обладающего именем и внутренними характеристиками. Доступ к данным объекта должен осуществляться с использованием механизма `friend`.

Требования к классу Персонажа

Класс Персонаж должен содержать:

- три свойства:
 - имя героя (тип строковый);
 - уровень здоровья (целочисленный тип);
 - уровень маны (целочисленный тип);
- все свойства должны иметь **закрытый модификатор доступа**;
- конструктор, инициализирующий значения всех свойств.

Использование публичных методов для получения или изменения значений свойств (`get`, `set`) **не допускается**.

Реализовать функцию вывода исходного состояния объекта, которая:

- принимает объект класса Персонаж по константной ссылке;
- выводит на экран имя героя, значение здоровья и значение маны;
- имеет доступ к закрытым членам класса Персонаж.

Реализовать класс Лекарь, который:

- содержит метод, изменяющий состояние объекта класса Персонаж:
 - увеличивает значение здоровья на 20;
 - увеличивает значение маны на 10;
- имеет доступ к закрытым членам класса Персонаж.

Алгоритм работы программы

1. Ввести с клавиатуры имя героя, начальное значение здоровья и начальное значение маны.

2. Создать объект класса Персонаж, используя введённые значения.
3. Вызвать функцию для вывода исходного состояния объекта.
4. Создать объект класса Лекарь.
5. Передать объект класса Персонаж в метод класса Лекарь для изменения его состояния.
6. Повторно вызвать функцию для вывода исходного состояния объекта.

Ограничения

- Прямой доступ к закрытым свойствам класса Персонаж вне дружественных сущностей запрещён.
- Использование публичных геттеров и сеттеров запрещено.
- Обращение к закрытым членам класса Персонаж из функции `main` запрещено.
- Использование механизма `friend` допускается только в рамках требований задания.

1.1 Описание входных данных

Значения через пробел: <Имя> <здоровье> <мана>

1.2 Описание выходных данных

Первая строка: "Hero: <Имя> | Health = <значение здоровья> | Mana = <значение маны>"

Вторая строка: "Hero: <Имя> | Health = <значение здоровья> | Mana = <значение маны>"

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `ch` класса `Character` предназначен для для описания объекта класса персонаж;
- объект `cin` класса `istream` предназначен для для ввода значений с клавиатуры;
- объект `healer` класса `Healer` предназначен для для описания объекта класса лекарь;
- функция `printValues` для вывода на экран имя героя, уровня здоровья и количества маны.

Класс `Character`:

- свойства/поля:
 - поле имя:
 - наименование — `name`;
 - тип — `string`;
 - модификатор доступа — `private`;
 - поле здоровье:
 - наименование — `hp`;
 - тип — `int`;
 - модификатор доступа — `private`;
 - поле мана:
 - наименование — `mana`;
 - тип — `int`;
 - модификатор доступа — `private`;
- функционал:
 - метод `Character` — конструктор с параметрами.

Класс Healer:

- функционал:
 - метод change — изменяет значения здоровья и маны.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Character			класс персонаж	
2	Healer			дружественный класс класса Character	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса **Character**

Функционал: конструктор с параметрами.

Параметры: string name - имя персонажа, int hp - здоровье персонажа, int mana - мана персонажа.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса *Character*

№	Предикат	Действия	№ перехода
1		инициализация значения всех свойств	Ø

3.2 Алгоритм метода **change** класса **Healer**

Функционал: изменяет значения здоровья и маны.

Параметры: Character& ch - ссылка на объект класса Character.

Возвращаемое значение: void, ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода *change* класса *Healer*

№	Предикат	Действия	№ перехода
1		увеличивает здоровье на 20 и ману на 10	Ø

3.3 Алгоритм функции printValues

Функционал: вывод на экран имени здоровья и маны персонажа.

Параметры: const Character& ch - ссылка на объект класса Character.

Возвращаемое значение: void, ничего не возвращает.

Алгоритм функции представлен в таблице 4.

Таблица 4 – Алгоритм функции printValues

№	Предикат	Действия	№ перехода
1		вывод на экран имени здоровья и маны персонажа	Ø

3.4 Алгоритм функции main

Функционал: основной алгоритм программы.

Параметры: нет.

Возвращаемое значение: целое - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 5.

Таблица 5 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		объявление переменной name типа string	2
2		объявление переменной hp типа int	3
3		объявление переменной mana типа int	4
4		ввод с клавиатуры переменных name, hp, mana	5
5		инициализация объекта ch класса Character с параметрами: name, hp, mana	6
6		вызов функции printValues с параметром ch	7
7		объявление объекта healer класса Healer	8
8		вызов метода change класса Healer с параметром ch	9

№	Предикат	Действия	№ перехода
9		вызов функции printValues с параметром ch	Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-2.

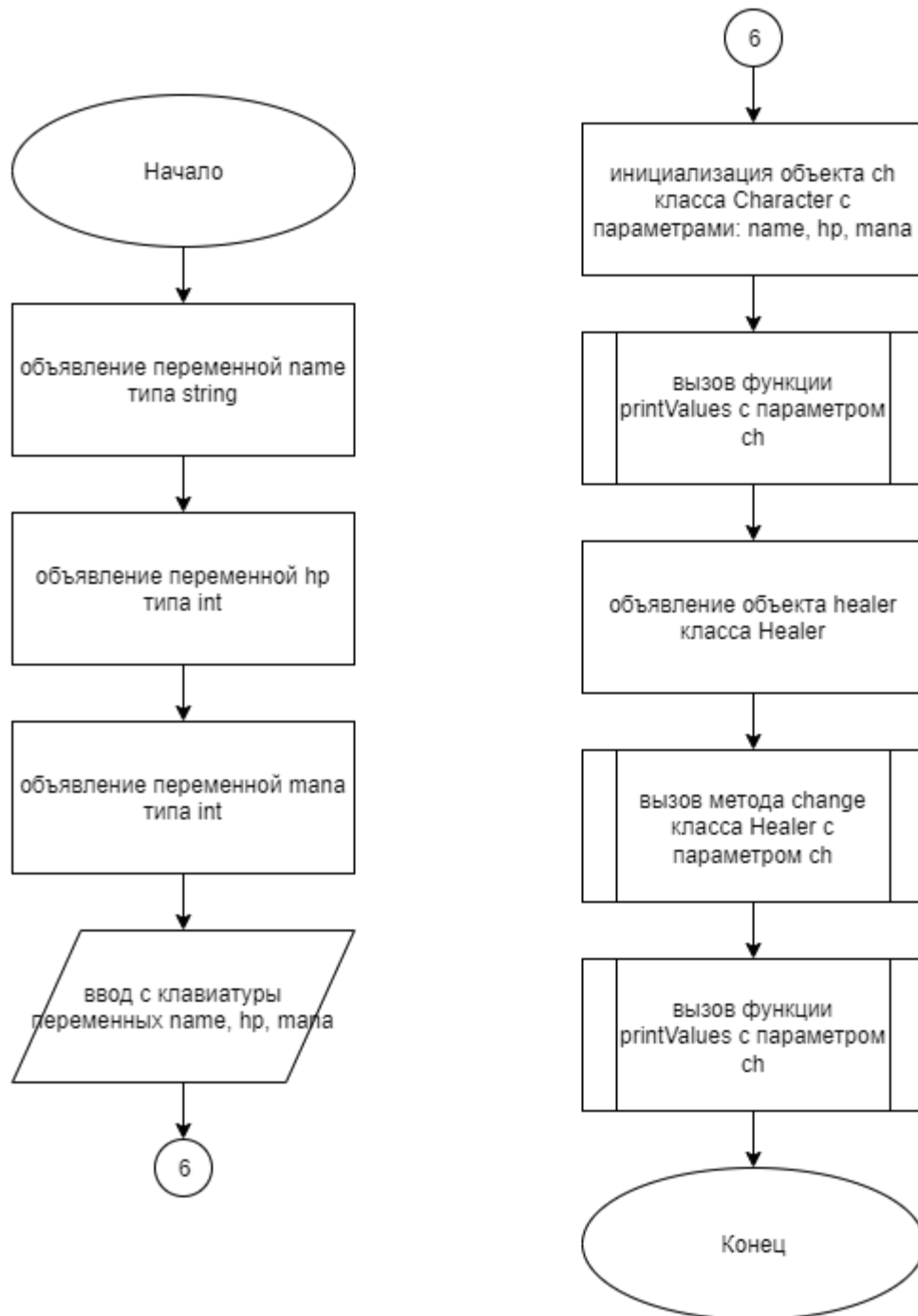


Рисунок 1 – Блок-схема алгоритма

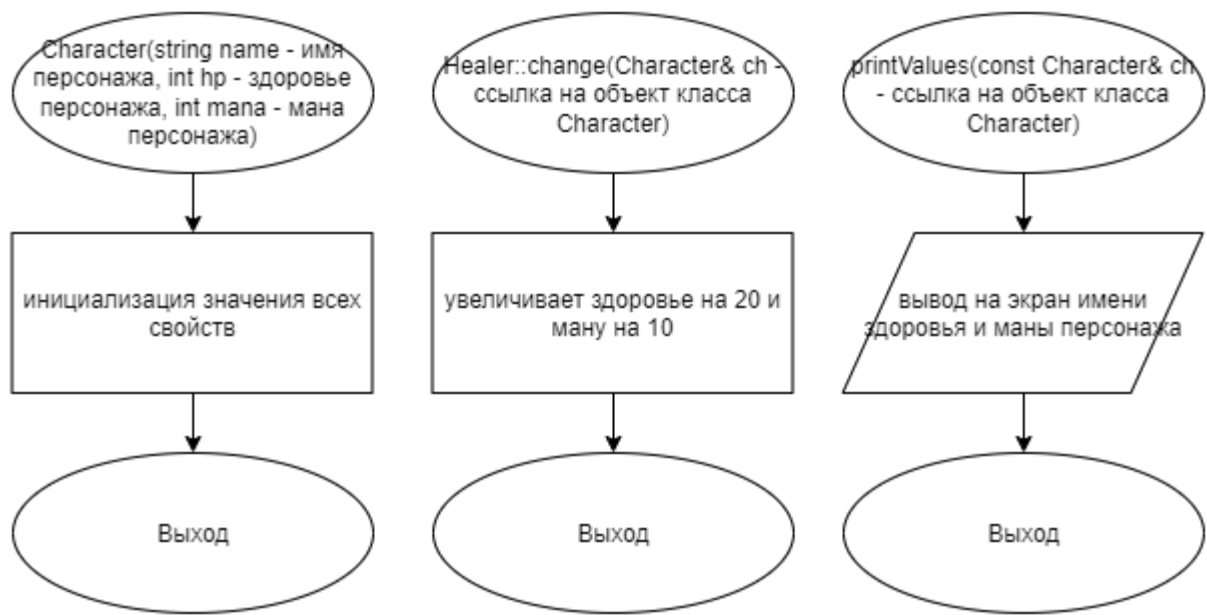


Рисунок 2 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Character.cpp

Листинг 1 – Character.cpp

```
#include "Character.h"
#include <string>

Character::Character(std::string n, int h, int m)
{
    name = n;
    hp = h;
    mana = m;
}

void printValues(const Character& ch)
{
    printf("Hero: %s | Health = %d | Mana = %d\n", ch.name.c_str(), ch.hp,
ch.mana);
}
```

5.2 Файл Character.h

Листинг 2 – Character.h

```
#ifndef __CHARACTER__H
#define __CHARACTER__H

#include <string>

class Healer;

class Character
{
private:
    std::string name;
    int hp;
    int mana;
}
```

```
public:
    Character(std::string n, int h, int m);
    friend void printValues(const Character& ch);
    friend class Healer;
};

#endif
```

5.3 Файл Healer.cpp

Листинг 3 – Healer.cpp

```
#include "Healer.h"
#include "Character.h"

void Healer::change(Character& ch)
{
    ch.hp += 20;
    ch.mana += 10;
}
```

5.4 Файл Healer.h

Листинг 4 – Healer.h

```
#ifndef __HEALER__H
#define __HEALER__H

#include "Character.h"

class Healer
{
public:
    void change(Character& ch);
};

#endif
```

5.5 Файл main.cpp

Листинг 5 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include <iostream>
#include "Character.h"
#include "Healer.h"

int main()
{
    std::string name;
    int hp;
    int mana;

    std::cin >> name >> hp >> mana;

    Character ch(name, hp, mana);

    printValues(ch);

    Healer healer;

    healer.change(ch);
    printValues(ch);

    return(0);
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 6.

Таблица 6 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Oleg 40 60	Hero: Oleg Health = 40 Mana = 60 Hero: Oleg Health = 60 Mana = 70	Hero: Oleg Health = 40 Mana = 60 Hero: Oleg Health = 60 Mana = 70
Maria 50 60	Hero: Maria Health = 50 Mana = 60 Hero: Maria Health = 70 Mana = 70	Hero: Maria Health = 50 Mana = 60 Hero: Maria Health = 70 Mana = 70
Vladislav 0 0	Hero: Vladislav Health = 0 Mana = 0 Hero: Vladislav Health = 20 Mana = 10	Hero: Vladislav Health = 0 Mana = 0 Hero: Vladislav Health = 20 Mana = 10

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).